TASK B · INHERIT AND IMPROVE

Task B — Read Me First

The four documents, and how to run the refactor's test suite

Inheriting a working but badly built codebase that serves real customers and cannot go down.

Live: https://crateful-handover.onrender.com

DOCUMENT	WHAT IT COVERS
Assessment	Nine findings ranked by harm, the cost of leaving each one, and the risk of the fix
Migration plan	Week 1, month 1, quarter 1 — every step shippable and reversible
The refactor	One handler rebuilt, with tests proving the behaviour survived and six defects did not
Standards	Ten machine-enforced rules, and how to get a resistant team to adopt them

The refactor runs

```bash
npm install
npm test
```

29 tests, no database and no network. Eight run against **both** the old and new implementation to show a customer cannot tell the difference — that is what makes it a refactor. The rest demonstrate the defects removed: no ownership check, SQL string concatenation, an unvalidated plan producing a `NaN` charge, a payment failure leaving the database ahead of billing, a retry double-charging, and a response leaking internal columns.

```
refactor/
├── before/change-plan.ts     the handler as inherited: 40 lines, six defects
├── after/pricing.ts          pure rules, imports nothing
├── after/subscriptions.ts    ownership, validation, one transaction, audit trail
├── after/route.ts            transport only
├── support/                  in-memory test doubles for Postgres, payments, mail
└── tests/                    behaviour.test.ts (both versions) + defects.test.ts
```

Building the site

```bash
npm run build     # docs/*.md -> dist/
```

The four documents are Markdown; `build/build.ts` renders them into the static site, so there is one source of truth rather than a copy that drifts.